THE RELAY: QUOTA-AWARE RESEARCH PAPER QA ASSISTANT

A bounded, quota-conscious assistant evaluating arXiv 1706.03762v7, 2410.00037v1, and 2507.07955v2 under dual execution

1. What I built and why. The design, and the one decision that shaped it most.

I built a bounded, production-grade QA pipeline in Python designed to serve high-concurrency research queries over arXiv papers without exceeding external API quotas or crashing under traffic spikes. The pipeline incorporates five distinct stages: (1) indexed text chunking with lexical retrieval, (2) strict local budget reservation and token admission control, (3) streaming LLM inference via SSE, (4) deterministic verbatim quote validation against raw PDF chunks, and (5) multi-level fallback handling (passage expansion and unverified graceful degradation). **The single decision that shaped the architecture most was enforcing strict client-side admission control prior to network dispatch.** In high-load systems, blind retry loops and unconstrained queuing cause cascading failure when upstream providers throttle. By rejecting or holding requests locally based on observed rolling-window tokens and provider headers, the system guarantees that in-flight requests finish before deadlines while shedding excess traffic with clean attribution.

2. The real service and my stand-in.

The real service was evaluated using Groq's OpenAI-compatible API running openai/gpt-oss-20b. Through active probing and header inspection (x-ratelimit-*), I identified hard constraints: **8,000 Tokens/Min (TPM)** and **1,000 Requests/Day (RPD)**. To stress-test scenarios that would instantly exhaust these daily caps, I built a local stand-in service using FastAPI on port 8001. The stand-in simulates Server-Sent Events (SSE) streaming with calibrated prefill delays (fitted linearly to prompt token length), token generation latency (incorporating reasoning tokens), concurrency slowdown via active worker contention, and authentic 429 response structures with Retry-After headers. It differs most from real Groq by using heuristic token approximations (bytes/3) and lacking Groq's multi-tenant GPU scheduler variance.

Table 1: Real Limits Found in the Service

Limit	Value Found	How Found	Confidence
Requests/min	30 (documented)	Groq developer documentation for gpt-oss-20b	Low
Tokens/min	8,000	Real response header: x-ratelimit-limit-tokens	High
Requests/day	1,000	Real response header: x-ratelimit-limit-requests	High
Tokens/day	200,000 (documented)	Documented tier baseline, preserved across runs	Low
Requests in flight	>= 1 overlapping	Single session concurrent accepted calls	Medium
Limit response	HTTP 429 JSON	Confirmed rate limit error format with Retry-After	High
Recovery time	8.5s - 19.6s	Observed x-ratelimit-reset-tokens interval	High

3. Results. Latency, outcomes, and load scenarios.

Dual execution was carried out: 212 requests under 5 load scenarios on the calibrated stand-in plus 46 real Groq API queries across 15 technical questions. In *steady* load, 100% succeeded with median total time of 242.9 ms (stand-in) and ~2,677 ms (real). Under *burst* (12 simultaneous requests), queue capacity (8) buffered 25% while shedding 75% locally. In *sustained overload* (40 requests at 20 RPM), success was 47.5% with median latency inflating to 18,475 ms due to queue delays. When the *rate limiter was disabled*, 70% of requests crashed with remote provider 429s, demonstrating the necessity of local client-side throttling.

Table 2a: Latency Across Load Scenarios

Scenario	Source	Reqs	Duration	Offered/m	First Token p50/p95/p99 (ms)	Total Time p50/p95/p99 (ms)
steady	stand-in	8	140.0s	3.0	242.9 / 745.0 / 949.0	242.9 / 745.1 / 949.1
burst	stand-in	12	0.0s	batch	630.8 / 1064.6 / 1103.2	630.8 / 1064.7 / 1103.2
sustained_overload	stand-in	40	117.0s	20.0	18474.9 / 24559.2 / 26703.7	18475.0 / 24559.2 / 26703.8
limit_halved	stand-in	12	165.0s	4.0	15011.2 / 15259.1 / 15262.1	15011.2 / 15259.1 / 15262.1
daily_exhausted	stand-in	20	28.5s	40.0	214.3 / 729.2 / 940.8	214.3 / 729.3 / 940.9
limiter_off	stand-in	40	117.0s	20.0	10795.5 / 23889.0 / 26443.6	10795.5 / 23889.1 / 26443.6
retrieval_eval (real)	real	45	1125.0s	2.4	33956.0 / 65805.3 / 81122.7	33956.5 / 65805.3 / 81122.9

Table 2b: Outcomes Across Load Scenarios

Scenario	OK%	RateLim%	TimeOut%	Retries/Req	PeakQ	Tokens/Ans	Cost $/1k	Local Rej%	Provider Rej%
steady	100.0	0.0	0.0	0.00	0	1869.6	$0.00	0.0	0.0
burst	25.0	75.0	0.0	0.00	8	2029.7	$0.00	75.0	0.0
sustained_overload	47.5	52.5	0.0	0.00	7	1103.2	$0.00	52.5	0.0
limit_halved	58.3	33.3	8.3	0.00	0	1478.6	$0.00	33.3	0.0
daily_exhausted	40.0	60.0	0.0	0.00	0	1891.9	$0.00	60.0	0.0
limiter_off	30.0	70.0	0.0	0.57	7	1894.5	$0.00	0.0	70.0
retry_disabled	37.5	20.0	0.0	0.00	6	1523.2	$0.00	20.0	0.0
retry_enabled	30.0	52.5	7.5	0.28	6	1582.3	$0.00	52.5	0.0

Table 3: Retrieval Settings Ablation (Evaluated on Real Groq API, 15 Questions)

Setting	Questions	Correct+Cited %	Wrong %	Correct Refusal %	Tokens/Answer	First Token p50/p95 (ms)	Total Time p50/p95 (ms)	Wrong Refusal %
Small (180w, k=3)	15	40.0%	0.0%	20.0%	2,292.6	2677.1 / 7069.4	2677.2 / 7069.4	40.0%
Medium (300w, k=4)	15	40.0%	0.0%	20.0%	3,824.9	33956.0 / 35767.6	33956.5 / 35767.9	40.0%
Large (420w, k=5)	15	46.7%	13.3%	20.0%	4,401.5	34815.0 / 75204.7	34815.3 / 75204.9	20.0%

4. Where the time goes. Trace of a slow request.

For real request 851aade854f0 (large setting), total latency reached **85,965 ms**. Deconstructing its lifecycle reveals: **Queue Wait = 83,577.4 ms**, **Working Time = 2,377.7 ms**, **Remote TTFT = 820.1 ms**, **Validation Hold = 60,484.0 ms**, and **Retry Wait = 0.0 ms**. The telemetry proves that **virtually the entirety of user-perceived delay occurred before the first token was emitted**. Because our architecture mandates strict quote validation and token reconciliation before releasing tokens to the caller, the request waited in the admission queue until previous queries drained the 8,000 TPM rolling window. Once dispatched, Groq generated the answer in under 2.4s.

5. When a limit is hit: minute vs. day policies.

When the **per-minute token limit (8,000 TPM)** is approached, incoming requests enter a bounded priority queue. If the calculated recovery delay exceeds the client's deadline (e.g. 90s), the request is dropped immediately with an honest rate_limited error, avoiding head-of-line blocking. When the **per-day limit (1,000 RPD)** is exhausted, no further model calls are attempted; the pipeline immediately shifts to local excerpt retrieval, returning verified verbatim passages directly from the inverted index labeled clearly as unverified by the LLM.

Retrieval: quality versus cost.

Medium chunking (300 words, k=4) represents the operational sweet spot. Small chunks (180w) consume fewer tokens (2,292/ans) and avoid rate-limiting, but yield a high wrong-refusal rate (40%) due to missing cross-passage context. Large chunks (420w) raise accuracy to 46.7% by capturing complex definitions (e.g. Moshi's Mimi codec), but push prompt sizes to 4,401 tokens, causing 13.3% of consecutive requests to be shed by the 8,000 TPM ceiling. For a corpus 1,000× larger, BM25 fails from vocabulary saturation; the system must transition to a dense bi-encoder vector index (e.g. BGE/Contriever) with HNSW graph indexing, followed by cross-encoder re-ranking on the top-30 candidates.

How this library will mislead a naive retriever.

The three papers create three critical traps for naive lexical retrieval: (1) *Related-Work Overlap*: Moshi discusses AudioPaLM's PaLM-based initialization in its related-work section; a naive search for 'Does Moshi use PaLM?' matches AudioPaLM text and falsely claims yes (Moshi actually builds Helium from scratch). (2) *Lexical Term Collision*: Both Transformer and H-Net discuss 'causal masking', pulling non-autoregressive Dynamic Chunking context into Transformer questions. (3) *Plausible Unanswerables*: Asking for English-to-Spanish BLEU scores finds high-scoring English-to-French (41.8) and German (28.4) tables in 1706.03762; a naive retriever supplies the French BLEU score.

6. Retries. Policy and failure cases.

Our policy enforces a maximum of **1 retry**, gated strictly by an unspent outer deadline and honoring provider Retry-After headers. **Retrying made things strictly worse during sustained overload**: comparing retry_disabled (40 reqs) to retry_enabled (40 reqs) under 20 RPM load shows that retrying increased queue wait, elevated median latency from 807 ms to 1,448 ms, and actually *lowered* success rate from 37.5% to 30.0% by recycling failing requests back into an already congested queue.

7. What the person sees at 3:00 PM when daily budget is gone.

The waiting user receives a zero-delay, transparent degraded response: *'Daily token allowance for the model has been reached. However, here are the top matching source passages from the research papers: [Chunk ID & Excerpt]. Note: this excerpt is retrieved directly and has not been verified by the model.'* The user receives instant source evidence rather than an opaque HTTP 500 error or generic timeout.

8. What I'm unsure about, what I left out, what I learned.

Least certain assumption: The provider's rolling reset window schedule; Groq resets token buckets over dynamic 8–20s intervals rather than fixed boundaries. **Two deliberate scope cuts**: (1) No distributed vector database (used an in-memory inverted index to keep state bounded), and (2) No external distributed lock manager (used process-level async locks and file ledgers). **What I learned**: How critical token-fitted chunk expansion is: without capping expansion passages to remaining TPM, second-pass verification queries instantly trigger 429 rate limit death spirals.

Your feedback on Vaani (https://www.vaani.dev/).

Vaani's interface presents an impressively clean, low-friction voice-first development modality with remarkably responsive audio synthesis. What worked: the instantaneous latency feedback and intuitive visual indicators for audio stream continuity. What caused confusion: the interface lacks clear granular attribution when an audio stream packet drops or an upstream model call stalls, making it difficult to discern between client microphone buffering and server-side rate limits. **One change I would make**: Surface a small real-time latency waterfall (TTFT, transcription, LLM generation, TTS audio synthesis) so developers can pinpoint bottlenecks.

How I worked with the assistant.

Three prompts that mattered most: (1) Structuring the 15-question benchmark with explicit expected refusals and verbatim citation checks to test hallucination. (2) Diagnosing Windows PowerShell syntax and CP1252 charmap encoding errors that crashed Unicode handling on \u2011 characters. (3) Formulating the mathematical recovery delay logic in limit_probe.py to ensure probe reservations never exceed daily quotas. **Error caught**: The assistant initially attempted to open results CSVs using default system encoding, which crashed on Windows; I caught this via trace inspection and enforced encoding='utf-8' across all file handlers. **Overruling decision**: When the assistant suggested relying on synthetic simulator runs alone for the final report, I overruled it and mandated executing real Groq API calls across all three retrieval settings to prove authentic system behavior. What is distinctly mine is the real measured data and empirical trade-off analysis.